Comunicación asíncrona accesible

Álvarez Flores José Julián

Desarrollo Web Profecional

José Miguel Carrera Pacheco

Tecnologías de la información

Tics

19/02/26

Reporte de Investigación Técnica

1. JavaScript Asíncrono (Async JavaScript)

JavaScript es, por naturaleza, un lenguaje de un solo hilo (single-threaded). Esto significa que solo puede ejecutar una tarea a la vez. Sin embargo, las aplicaciones modernas necesitan realizar tareas pesadas (como pedir datos a un servidor) sin congelar la interfaz del usuario. Aquí entra la programación asíncrona.

Conceptos Clave

Event Loop: Mecanismo que permite a JavaScript realizar operaciones no bloqueantes, delegando tareas al navegador y ejecutándolas cuando terminan.

Promesas (Promises): Objetos que representan la terminación (o falla) eventual de una operación asíncrona. Tienen tres estados: pendiente, cumplida o rechazada.

Async / Await: Azúcar sintáctico introducido en ES2017 que permite escribir código asíncrono que se lee y comporta como código síncrono, mejorando drásticamente la legibilidad.

Impacto en el sistema: Permite que la aplicación siga respondiendo al usuario (hacer scroll, clics) mientras espera datos externos, gestionando la latencia de forma transparente.

2. APIs REST (Representational State Transfer)

REST es un estilo de arquitectura para diseñar servicios web. Funciona como un contrato de comunicación entre el cliente (frontend) y el servidor (backend) utilizando el protocolo HTTP estándar.

Principios Fundamentales

Una API RESTful se basa en recursos (como "Usuarios", "Productos") accesibles mediante URLs específicas y manipulables a través de verbos HTTP:

Método HTTP	Acción	Uso Típico
GET	Leer	Obtener datos del servidor.
POST	Crear	Enviar nuevos datos para ser guardados.
PUT / PATCH	Actualizar	Modificar datos existentes.
DELETE	Borrar	Eliminar un recurso.

Impacto en el sistema: Estandariza la forma de trabajar con datos externos, desacoplando el frontend del backend y permitiendo que ambos evolucionen independientemente.

3. Estados de Carga Accesibles

Dado que las peticiones a APIs externas implican latencia, la interfaz debe comunicar explícitamente al usuario que "algo está sucediendo". Un estado de carga accesible no es solo visual, sino que debe ser perceptible por tecnologías de asistencia (como lectores de pantalla).

Requisitos de Accesibilidad (WCAG)

- Indicadores Visuales: Usar "spinners", barras de progreso o "esqueletos" (skeleton screens) para reducir la percepción de espera.
- Atributos ARIA: Uso de `aria-busy="true"` en la sección que se está actualizando y `role="status"` o `aria-live="polite"` para anunciar cambios.
- Gestión del Foco: Evitar que el usuario interactúe con elementos que aún no están listos (ej. deshabilitar botones de envío mientras se procesa).

Impacto en el sistema: Mejora la experiencia de usuario (UX) durante los periodos de latencia, evitando la frustración de pensar que el sistema se ha congelado.

4. Manejo de Errores de Red

En un entorno web, los fallos son inevitables: el usuario puede perder la conexión, el servidor puede caerse o la API puede devolver datos incorrectos. Una aplicación robusta no debe romperse ante estos eventos, sino degradarse elegantemente.

Estrategias de Mitigación

1. Bloques Try/Catch: Estructura básica para capturar errores en tiempo de ejecución y evitar que la aplicación se detenga por completo.
2. Feedback al Usuario: Mostrar mensajes claros y accionables (ej. "No pudimos guardar tus cambios. Revisa tu conexión") en lugar de códigos técnicos.
3. Lógica de Reintento (Retries): Implementar mecanismos para reintentar automáticamente una petición fallida tras unos segundos, útil para cortes de red intermitentes.
4. Validación de Respuestas HTTP: No asumir que toda respuesta es exitosa. Verificar códigos de estado (ej. 404 Not Found, 500 Server Error) antes de procesar datos.

Impacto en el sistema: Permite que el software sobreviva a fallos y condiciones adversas, manteniendo la confianza del usuario y la integridad de los datos.